

实验三：MongoDB数据库的文档基本操作实验指导

【实验名称】 实验三：MongoDB数据库的文档基本操作
【实验目的】 1. 掌握 MongoDB 插入文档 2. 掌握 MongoDB 更新文档 3. 掌握 MongoDB 删除文档 4. 掌握 MongoDB 删除集合
【实验原理】 MongoDB 是一个介于关系数据库和非关系数据库之间的产品，是非关系数据库当中功能最丰富，最像关系数据库的。 文档的数据结构和 JSON 基本一样。 所有存储在集合中的数据都是 BSON 格式。 BSON 是一种类 json 的一种二进制形式的存储格式,简称 Binary JSON。
【实验环境】 OS: Ubuntu16.04 or Windows 7/10 MongoDB: v3.6 from l9000 or v4.2.19
【实验步骤】 题目一：插入文档 MongoDB 使用 insert() 或 save() 方法向集合中插入文档，语法如下： db.COLLECTION_NAME.insert(document) 示例：向数据库 student 的 stuinfo 集合插入以下数据

_id	name	age
001	alice	18
002	nancy	19
003	harry	18
004	curry	19

//insert 方法插入单个文档

```
db.stuinfo.insert({_id:001,name:'alice',age:18})
```

//save 方法插入单个文档

```
db.stuinfo.save({_id:002,name:'nancy',age:19})
```

//插入多个文档

```
db.stuinfo.insert([{_id:003,name:'harry',age:18},{_id:004,name:'curry',age:19}])
```

文档插入完成后，使用 find () 查看集合数据，具体操作步骤如下图所示：

```
> use student
switched to db student
> db.stuinfo.insert({_id:001,name:'alice',age:18})
WriteResult({ "nInserted" : 1 })
> db.stuinfo.save({_id:002,name:'nancy',age:19})
WriteResult({ "nMatched" : 0, "nUpserted" : 1, "nModified" : 0, "_id" : 2 })
> db.stuinfo.insert([{_id:003,name:'harry',age:18},{_id:004,name:'curry',age:19}])
BulkWriteResult({
  "writeErrors" : [ ],
  "writeConcernErrors" : [ ],
  "nInserted" : 2,
  "nUpserted" : 0,
  "nMatched" : 0,
  "nModified" : 0,
  "nRemoved" : 0,
  "upserted" : [ ]
})
> db.stuinfo.find()
{ "_id" : 1, "name" : "alice", "age" : 18 }
{ "_id" : 2, "name" : "nancy", "age" : 19 }
{ "_id" : 3, "name" : "harry", "age" : 18 }
{ "_id" : 4, "name" : "curry", "age" : 19 }
>
```

以上示例中 stuinfo 是我们的集合名，如果该集合不在该数据库中，MongoDB 会自动创建该集合并插入文档。

还可以先将文档定义为一个变量，再进行插入：

```
> s={_id:5,name:'张三',age:19}
```

```
> db.stuinfo.insert(s)
```

操作结果如下图所示:

```
> s={_id:5,name:'张三',age:19}
{ "_id" : 5, "name" : "张三", "age" : 19 }
> db.stuinfo.insert(s)
WriteResult({ "nInserted" : 1 })
> db.stuinfo.find()
{ "_id" : 1, "name" : "alice", "age" : 18 }
{ "_id" : 2, "name" : "nancy", "age" : 19 }
{ "_id" : 3, "name" : "harry", "age" : 18 }
{ "_id" : 4, "name" : "curry", "age" : 19 }
{ "_id" : 5, "name" : "张三", "age" : 19 }
```

题目二：更新文档（区分update和save的区别）

MongoDB 使用 update() 和 save() 方法来更新集合中的文档。

(1) update() 方法用于更新已存在的文档。语法格式如下:

```
db.collection.update (<criteria>, <objNew>, upsert, multi, writeConcern)
```

参数说明:

query : update 的查询条件, 类似 sql update 查询内 where 后面的。

objNew : update 的对象和一些更新的操作符 (如\$,\$inc...) 等, 也可以理解为 sql update 查询内 set 后面的

upsert : 可选, 这个参数的意思是, 如果不存在 update 的记录, 是否插入 objNew,true 为插入, 默认是 false, 不插入。

multi : 可选, mongodb 默认是 false,只更新找到的第一条记录, 如果这个参数为 true,就把按条件查出来多条记录全部更新。

writeConcern :可选, 抛出异常的级别。

示例

通过 update() 方法来更新上述文档的姓名(name): 将姓名(name)为 curry 的文档更新为了 "jack"。

执行命令：db.stuinfo.update({name:'curry'},{\$set:{name:'王五'}})

通过 find () 查看修改是否成功，具体操作步骤如下所示：

```
> db.stuinfo.find()
{ "_id" : 1, "name" : "alice", "age" : 18 }
{ "_id" : 2, "name" : "nancy", "age" : 19 }
{ "_id" : 3, "name" : "harry", "age" : 18 }
{ "_id" : 4, "name" : "curry", "age" : 19 }
{ "_id" : 5, "name" : "张三", "age" : 19 }
> db.stuinfo.update({name:'curry'},{$set:{name:'李四'}})
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
> db.stuinfo.find()
{ "_id" : 1, "name" : "alice", "age" : 18 }
{ "_id" : 2, "name" : "nancy", "age" : 19 }
{ "_id" : 3, "name" : "harry", "age" : 18 }
{ "_id" : 4, "name" : "李四", "age" : 19 }
{ "_id" : 5, "name" : "张三", "age" : 19 }
```

以上语句只会修改第一条发现的文档，如果要修改多条相同的文档，则需要设置 multi 参数为 true。

db.stuinfo.update({name:'nancy'},{\$set:{name:'王五'}},false,true)

操作结果如下图所示：

```
> db.stuinfo.find()
{ "_id" : 1, "name" : "alice", "age" : 18 }
{ "_id" : 2, "name" : "nancy", "age" : 19 }
{ "_id" : 3, "name" : "harry", "age" : 18 }
{ "_id" : 4, "name" : "李四", "age" : 19 }
{ "_id" : 5, "name" : "张三", "age" : 19 }
{ "_id" : 6, "name" : "nancy", "age" : 19 }
{ "_id" : 7, "name" : "nancy", "age" : 18 }
> db.stuinfo.update({name:'nancy'},{$set:{name:'王五'}},false,true)
WriteResult({ "nMatched" : 3, "nUpserted" : 0, "nModified" : 3 })
> db.stuinfo.find()
{ "_id" : 1, "name" : "alice", "age" : 18 }
{ "_id" : 2, "name" : "王五", "age" : 19 }
{ "_id" : 3, "name" : "harry", "age" : 18 }
{ "_id" : 4, "name" : "李四", "age" : 19 }
{ "_id" : 5, "name" : "张三", "age" : 19 }
{ "_id" : 6, "name" : "王五", "age" : 19 }
{ "_id" : 7, "name" : "王五", "age" : 18 }
```

(2) save() 方法通过传入的文档来替换已有文档。语法格式如下：

db.collection.save (<document>,{writeConcern:<document>})

参数说明：

document : 文档数据。

writeConcern :可选，抛出异常的级别。

示例：替换 _id 为 2 的文档的数据：

db.stuinfo.save({_id:2,name:'curry',age:20})

操作步骤如下图所示：

```
> db.stuinfo.find().pretty()
{ "_id" : 1, "name" : "alice", "age" : 18 }
{ "_id" : 2, "name" : "赵六", "age" : 20 }
{ "_id" : 3, "name" : "harry", "age" : 18 }
{ "_id" : 4, "name" : "李四", "age" : 19 }
{ "_id" : 5, "name" : "张三", "age" : 19 }
{ "_id" : 6, "name" : "王五", "age" : 19 }
{ "_id" : 7, "name" : "王五", "age" : 18 }
> db.stuinfo.save({ _id:2,name:'curry',age:20})
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
> db.stuinfo.find().pretty()
{ "_id" : 1, "name" : "alice", "age" : 18 }
{ "_id" : 2, "name" : "curry", "age" : 20 }
{ "_id" : 3, "name" : "harry", "age" : 18 }
{ "_id" : 4, "name" : "李四", "age" : 19 }
{ "_id" : 5, "name" : "张三", "age" : 19 }
{ "_id" : 6, "name" : "王五", "age" : 19 }
{ "_id" : 7, "name" : "王五", "age" : 18 }
```

更多示例: (选做)

```
for(var i=1;i<10;i++) db.col.insert({count:i,test2:false,test5:true})
```

只更新第一条记录:

```
db.col.update( { "count" : { $gt : 1 } } , { $set : { "test2" : "OK" } } );
```

全部更新:

```
db.col.update( { "count" : { $gt : 3 } } , { $set : { "test2" : "OK" } },false,true );
```

只添加第一条:

```
db.col.update( { "count" : { $gt : 4 } } , { $set : { "test5" : "OK" } },true,false );
```

全部添加加进去:

```
db.col.update( { "count" : { $gt : 5 } } , { $set : { "test5" : "OK" } },true,true );
```

全部更新:

```
db.col.update( { "count" : { $gt : 15 } } , { $inc : { "count" : 1 } },false,true );
```

只更新第一条记录:

```
db.col.update( { "count" : { $gt : 10 } } , { $inc : { "count" : 1 } },false,false );
```

题目三: 删除文档(注意区分remove和delete)

remove(), deleteOne() 和 deleteMany() 方法可以用来移除集合中的数据。

(1) 删除集合 col 下全部文档

```
db.col.deleteMany({})
```

或 `db.col.remove({})`

具体如下图所示

```
> db.col.find()
{ "_id" : ObjectId("5c1361e7d7497c8691fbba7f"), "count" : 1, "test2" : false, "test5" : true }
{ "_id" : ObjectId("5c1361e7d7497c8691fbba80"), "count" : 2, "test2" : false, "test5" : true }
{ "_id" : ObjectId("5c1361e7d7497c8691fbba81"), "count" : 3, "test2" : false, "test5" : true }
{ "_id" : ObjectId("5c1361e7d7497c8691fbba82"), "count" : 4, "test2" : false, "test5" : true }
{ "_id" : ObjectId("5c1361e7d7497c8691fbba83"), "count" : 5, "test2" : false, "test5" : true }
{ "_id" : ObjectId("5c1361e7d7497c8691fbba84"), "count" : 6, "test2" : false, "test5" : true }
{ "_id" : ObjectId("5c1361e7d7497c8691fbba85"), "count" : 7, "test2" : false, "test5" : true }
{ "_id" : ObjectId("5c1361e7d7497c8691fbba86"), "count" : 8, "test2" : false, "test5" : true }
{ "_id" : ObjectId("5c1361e7d7497c8691fbba87"), "count" : 9, "test2" : false, "test5" : true }
> db.col.deleteMany({})
{ "acknowledged" : true, "deletedCount" : 9 }
> db.col.find()
> for(var i=1;i<10;i++) db.col.insert({count:i,test2:false,test5:true})
WriteResult({ "nInserted" : 1 })
> db.col.find()
{ "_id" : ObjectId("5c136290d7497c8691fbba88"), "count" : 1, "test2" : false, "test5" : true }
{ "_id" : ObjectId("5c136290d7497c8691fbba89"), "count" : 2, "test2" : false, "test5" : true }
{ "_id" : ObjectId("5c136290d7497c8691fbba8a"), "count" : 3, "test2" : false, "test5" : true }
{ "_id" : ObjectId("5c136290d7497c8691fbba8b"), "count" : 4, "test2" : false, "test5" : true }
{ "_id" : ObjectId("5c136290d7497c8691fbba8c"), "count" : 5, "test2" : false, "test5" : true }
{ "_id" : ObjectId("5c136290d7497c8691fbba8d"), "count" : 6, "test2" : false, "test5" : true }
{ "_id" : ObjectId("5c136290d7497c8691fbba8e"), "count" : 7, "test2" : false, "test5" : true }
{ "_id" : ObjectId("5c136290d7497c8691fbba8f"), "count" : 8, "test2" : false, "test5" : true }
{ "_id" : ObjectId("5c136290d7497c8691fbba90"), "count" : 9, "test2" : false, "test5" : true }
> db.col.remove({})
WriteResult({ "nRemoved" : 9 })
> db.col.find()
>
```

(2)删除指定条件的文档

删除集合 `stuinfo` 中 `name` 等于王五的全部文档:

`db.stuinfo.deleteMany({name:'王五'})`

```
> db.stuinfo.find()
{ "_id" : 1, "name" : "alice", "age" : 18 }
{ "_id" : 2, "name" : "curry", "age" : 20 }
{ "_id" : 3, "name" : "harry", "age" : 18 }
{ "_id" : 4, "name" : "李四", "age" : 19 }
{ "_id" : 5, "name" : "张三", "age" : 19 }
{ "_id" : 6, "name" : "王五", "age" : 19 }
{ "_id" : 7, "name" : "王五", "age" : 18 }
> db.stuinfo.deleteMany({name:'王五'})
{ "acknowledged" : true, "deletedCount" : 2 }
> db.stuinfo.find()
{ "_id" : 1, "name" : "alice", "age" : 18 }
{ "_id" : 2, "name" : "curry", "age" : 20 }
{ "_id" : 3, "name" : "harry", "age" : 18 }
{ "_id" : 4, "name" : "李四", "age" : 19 }
{ "_id" : 5, "name" : "张三", "age" : 19 }
```

删除 `age` 等于 18 的一个文档:

`db.stuinfo.deleteOne({age:18})`

```
> db.stuinfo.find()
{ "_id" : 1, "name" : "alice", "age" : 18 }
{ "_id" : 2, "name" : "curry", "age" : 20 }
{ "_id" : 3, "name" : "harry", "age" : 18 }
{ "_id" : 4, "name" : "李四", "age" : 19 }
{ "_id" : 5, "name" : "张三", "age" : 19 }
> db.stuinfo.deleteOne({age:18})
{ "acknowledged" : true, "deletedCount" : 1 }
> db.stuinfo.find()
{ "_id" : 2, "name" : "curry", "age" : 20 }
{ "_id" : 3, "name" : "harry", "age" : 18 }
{ "_id" : 4, "name" : "李四", "age" : 19 }
{ "_id" : 5, "name" : "张三", "age" : 19 }
>
```